ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN - ĐIỆN TỬ

KIẾN TRÚC MÁY TÍNH

MILESTONE 3

DESIGN OF A PIPELINED RISC-V PROCESSOR

GVHD: PGS.TS. TRẦN HOÀNG LINH
Lớp: KTMT_L02
SV thực hiện: Nguyễn Phương Ngân - 2312229
Trần Nguyễn Trung Trực - 2313720
Trần Đức Hoàng - 2311076

TP Hồ Chí Minh, Tháng 1 Năm 2026.

Mục lục

1 INTRODUCTION **2**

2 DESIGN STRATEGY **2**
2.1 Overview . . . 2
2.1.1 Datapath . . . 2
2.1.2 Specification . . . 3
2.2 Model 1: Non-forwarding . . . 4
2.3 Model 2: Forwarding . . . 5
2.4 Model 3: Forwarding with Always-Taken . . . 6
2.5 Model 4: Forwarding with 2-bit Scheme Prediction . . . 7
2.6 Model 5: Forwarding with PC-indexed 2-bit Prediction . . . 8

3 VERIFICATION STRATEGY **9**
3.1 Kiểm tra hoạt động CPU với từng câu lệnh . . . 9
3.2 Kiểm tra IPC và Misprediction với Benchmarks . . . 9

4 RESULT **10**
4.1 Quartus Synthesis Results . . . 10
4.2 Performance Analysis . . . 11
4.2.1 Nhận xét về IPC (Instructions Per Cycle) . . . 11
4.2.2 Nhận xét về Branch Misprediction Rate . . . 11
4.2.3 Tổng hợp và Kết luận . . . 11

5 Applications **12**

Tài liệu tham khảo **14**

Danh sách hình vẽ

1 Datapath của CPU Pipeline 5 tầng IF-ID-EX-MEM-WB . . . 2
2 Datapath của Model 1 - Pipeline Non-forwarding . . . 4
3 Datapath của Model 2 - Pipeline với Forwarding . . . 5
4 Datapath của Model 3 - Pipeline với Forwarding và Always-Taken Prediction . . . 6
5 Datapath của Model 4 - Pipeline với Forwarding và 2-bit Prediction . . . 7
6 Datapath của Model 5 - Forwarding with PC-indexed 2-bit Prediction . . . 8
7 Kết quả pass testcode . . . 12
8 Ứng dụng LED7D . . . 13
9 SW 17 được bật lên reset LED . . . 13

1 INTRODUCTION

Trong xu hướng phần cứng mã nguồn mở ngày càng phát triển, RISC-V trở thành một ISA nổi bật nhờ thiết kế gọn, dễ mở rộng và phù hợp cho nhiều mục tiêu (giáo dục, nhúng, nghiên cứu). Ở milestone này, nhóm tập trung xây dựng CPU RV32I dạng pipeline 5 tầng, nhằm cải thiện thông lượng thực thi so với thiết kế single-cycle trước đó.

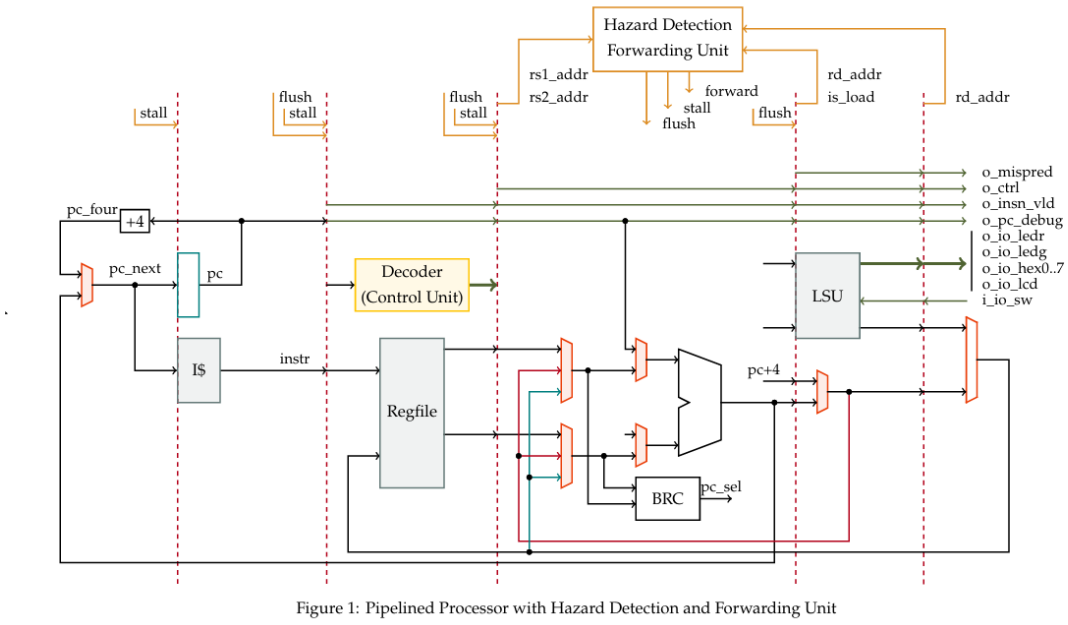
Thiết kế được chia thành các khối chức năng (fetch/decode/execute/memory/write-back), đi kèm cơ chế xử lý data hazard và control hazard. Nhóm cũng triển khai nhiều model pipeline như: non-forwarding, forwarding, always-taken, và 2-bit scheme prediction để đánh giá tác động đến hiệu năng (IPC) và tỷ lệ đoán sai (misprediction).

2 DESIGN STRATEGY

2.1 Overview

Milestone 3 tập trung vào việc chuyển đổi thiết kế CPU RV32I từ kiến trúc single-cycle (Milestone 2) sang kiến trúc pipeline 5 tầng IF–ID–EX–MEM–WB. Bằng cách phân rã quá trình thực thi lệnh thành các giai đoạn và cho phép nhiều lệnh được xử lý đồng thời ở các stage khác nhau, kiến trúc pipeline hướng tới mục tiêu tăng thông lượng (throughput) và cải thiện hiệu năng tổng thể.

2.1.1 Datapath



Hình 1: *Datapath của CPU Pipeline 5 tầng IF-ID-EX-MEM-WB*

Hình 1 minh họa kiến trúc datapath hoàn chỉnh của CPU pipeline 5 tầng. Datapath được tổ chức thành các stage rõ ràng với các pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB)

ngăn cách giữa các tầng. Mỗi stage thực hiện một chức năng cụ thể: IF (Instruction Fetch) nạp lệnh từ bộ nhớ, ID (Instruction Decode) giải mã và đọc thanh ghi, EX (Execute) thực hiện phép toán ALU hoặc tính địa chỉ, MEM (Memory Access) truy cập bộ nhớ dữ liệu, và WB (Write Back) ghi kết quả về thanh ghi.

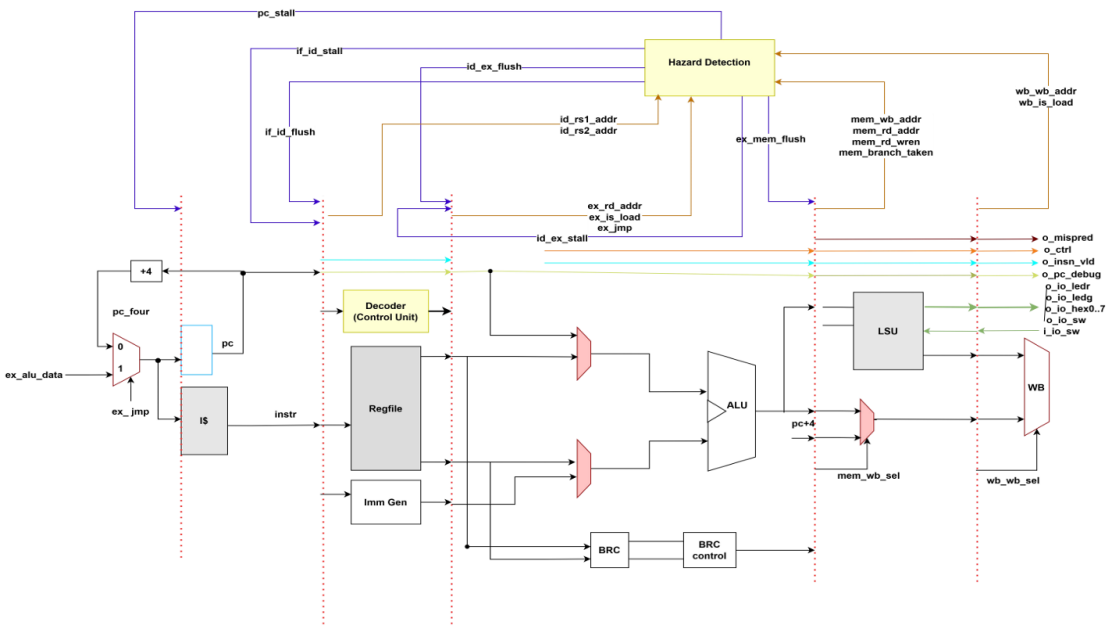
2.1.2 Specification

Tên tín hiệu	**Độ rộng**	**Hướng**	**Mô tả**
i_clk	1	input	Xung clock toàn cục, kích hoạt cạnh lên
i_reset	1	input	Tín hiệu reset toàn cục, active-low
o_pc_debug	32	output	Program counter để debug
o_insn_vld	1	output	Tín hiệu instruction hợp lệ
o_ctrl	1	output	Lệnh chuyển luồng điều khiển
o_mispred	1	output	Tín hiệu dự đoán sai nhánh
o_io_ledr	32	output	Xuất dữ liệu ra LED đỏ
o_io_ledg	32	output	Xuất dữ liệu ra LED xanh
o_io_hex0...7	7	output	Xuất dữ liệu ra LED 7 đoạn
o_io_lcd	32	output	Xuất dữ liệu ra thanh ghi LCD
i_io_sw	32	input	Nhận dữ liệu từ công tắc (switches)

Bảng 1: Đặc tả tín hiệu I/O của CPU

Bảng trên liệt kê đầy đủ các tín hiệu input/output của CPU. Các tín hiệu được chia thành 3 nhóm chính: (1) Tín hiệu điều khiển cơ bản (clock, reset), (2) Tín hiệu debug và monitoring (pc_debug, insn_vld, ctrl, mispred), và (3) Tín hiệu I/O ngoại vi (LED, 7-segment, LCD, switches) để tương tác với kit FPGA.

2.2 Model 1: Non-forwarding



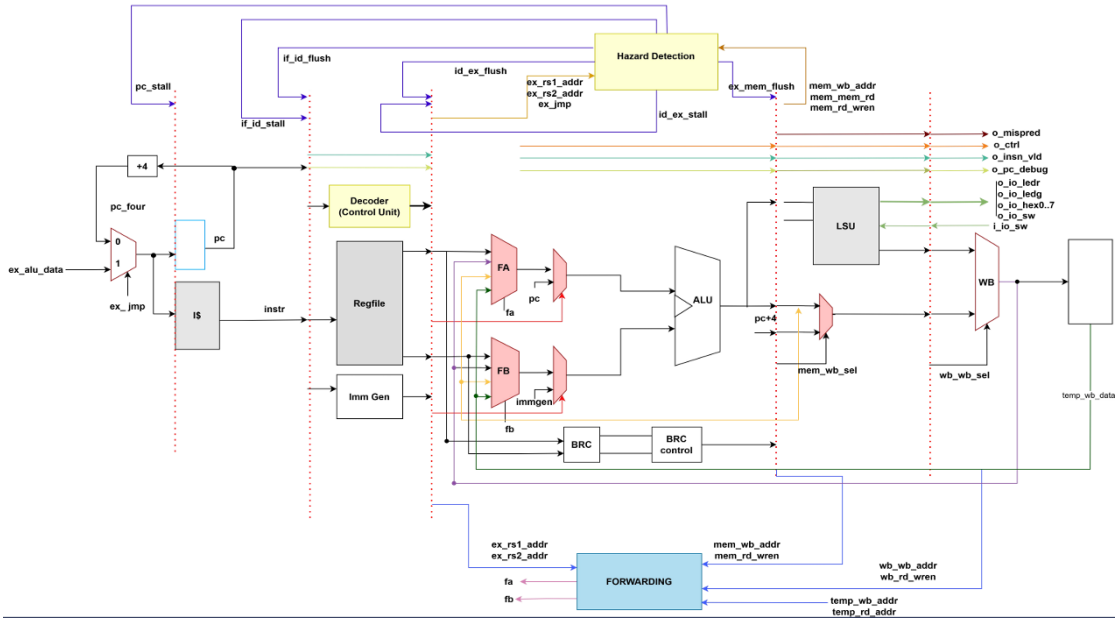
Hình 2: *Datapath của Model 1 - Pipeline Non-forwarding*

Trong mô hình non-forwarding này, CPU được tổ chức theo pipeline 5 tầng và các tín hiệu stall/flush được điều khiển tập trung bởi khối Hazard Detection. Ở data hazard, khối này kiểm tra sự phụ thuộc thanh ghi giữa lệnh đang ở ID (`id_rs1_addr`, `id_rs2_addr`) với thanh ghi đích của các lệnh ở các tầng sau (`ex_rd_addr`, `mem_rd_addr`, `wb_rd_addr`), đồng thời loại trừ x0 và tận dụng tín hiệu `id_I_type` để không so sánh rs2 với các lệnh I-type (vì không dùng rs2). Khi phát hiện hazard, hệ thống stall PC và IF/ID (`pc_stall` = `if_id_stall` = 1) để giữ nguyên lệnh đang decode, đồng thời flush ID/EX (`id_ex_flush` = 1) nhằm chèn bubble vào stage EX cho đến khi an toàn.

Với control hazard, CPU áp dụng hướng predict-not-taken theo đúng datapath: mặc định `PC_next` = PC + 4, còn việc nhảy/rẽ nhánh chỉ được quyết định ở EX. Tại EX, `brc_control` so sánh toán hạng để quyết định nhánh theo funct3, và tín hiệu nhảy tổng `ex_jmp` được tạo từ (branch taken) hoặc (JAL/JALR). Nếu `ex_jmp` = 1, PC được redirect sang địa chỉ mục tiêu từ ALU (`pc_next` = `ex_alu_data`) và hazard unit sẽ flush IF/ID + ID/EX để loại bỏ các lệnh đã nạp theo đường PC+4 (bubble do đổi luồng).

Ngoài ra, phần instruction fetch được tích hợp trong LSU: `o_instr` (đưa sang ID) là một thanh ghi cập nhật theo clock, và cũng tuân theo stall/flush (stall thì giữ nguyên instruction, flush thì đẩy về 0 như NOP). Các pipeline register của nhóm dùng reset active-low bất đồng bộ và flush đồng bộ, trong đó flush có ưu tiên cao hơn stall. Regfile đọc tổ hợp và ghi đồng bộ, đảm bảo hành vi chuẩn cho pipeline non-forwarding.

2.3 Model 2: Forwarding



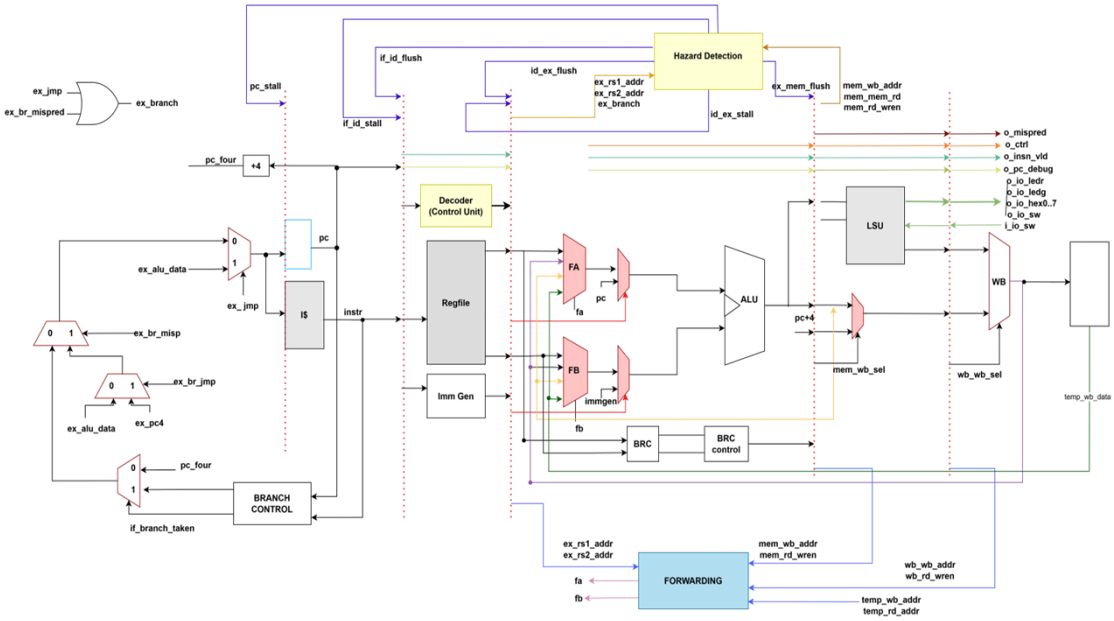
Hình 3: *Datapath của Model 2 - Pipeline với Forwarding*

Trong model 2, nhóm triển khai pipeline có forwarding để giảm stall do RAW data hazard. Cụ thể, khối `forwarding_unit` so sánh `ex_rs1_addr`/`ex_rs2_addr` với `rd` ở các stage sau (`mem_wb_addr`, `wb_wb_addr`, và một bản sao trễ `temp_wb_addr`) kèm tín hiệu `reg_write`, từ đó tạo hai tín hiệu chọn `forward_a`/`forward_b`.

Ở stage EX, hai toán hạng `fw_a`, `fw_b` được tạo bằng mux 4-ngõ: lấy trực tiếp từ `ex_rs_data` hoặc bypass từ MEM (`mem_alu_pc_data`), WB (`wb_wb_data`), hoặc đường temp (`temp_wb_data`). Hai giá trị này vừa được dùng cho ALU (tạo `ex_alu_data`) vừa được đưa vào `brc_control` để so sánh nhánh, đảm bảo cả phép tính lẫn quyết định branch đều dùng dữ liệu mới nhất. Ngoài ra, dữ liệu store cũng được "đi đường tắt"bằng cách pipeline `fw_b` sang `mem_rs2_data`, giúp tránh hazard khi giá trị cần store vừa được tính ở lệnh trước.

Về hazard, do đã có forwarding nên `hazard_detection` của nhóm chỉ stall trong trường hợp load-use: nếu lệnh load đang ở MEM (`mem_is_load`=1, `mem_rd_wren`=1) và thanh ghi đích `mem_rd_addr` trùng với `ex_rs1_addr` hoặc `ex_rs2_addr` (khác x0), hệ thống sẽ bật `pc_stall`/ `if_id_stall`/ `id_ex_stall` để giữ nguyên tiến trình, đồng thời kích hoạt `ex_mem_flush` nhằm chèn bubble ở ranh giới EX→MEM. Với control hazard, nhóm vẫn theo hướng predict-not-taken (mặc định `PC_next` = PC+4), và chỉ khi EX kết luận có chuyển luồng (`ex_jmp`=1) thì PC được redirect sang `ex_alu_data`; đồng thời `hazard_detection` phát `if_id_flush` và `id_ex_flush` để loại bỏ các lệnh đã nạp sai đường.

2.4 Model 3: Forwarding with Always-Taken



Hình 4: *Datapath của Model 3 - Pipeline với Forwarding và Always-Taken Prediction*

Model 3 giữ nguyên datapath có forwarding như Model 2, nhưng bổ sung cơ chế dự đoán nhánh "always-taken"ngay từ IF. Branch control đọc instruction và tính trước branch target, giảm penalty khi branch thực sự taken.

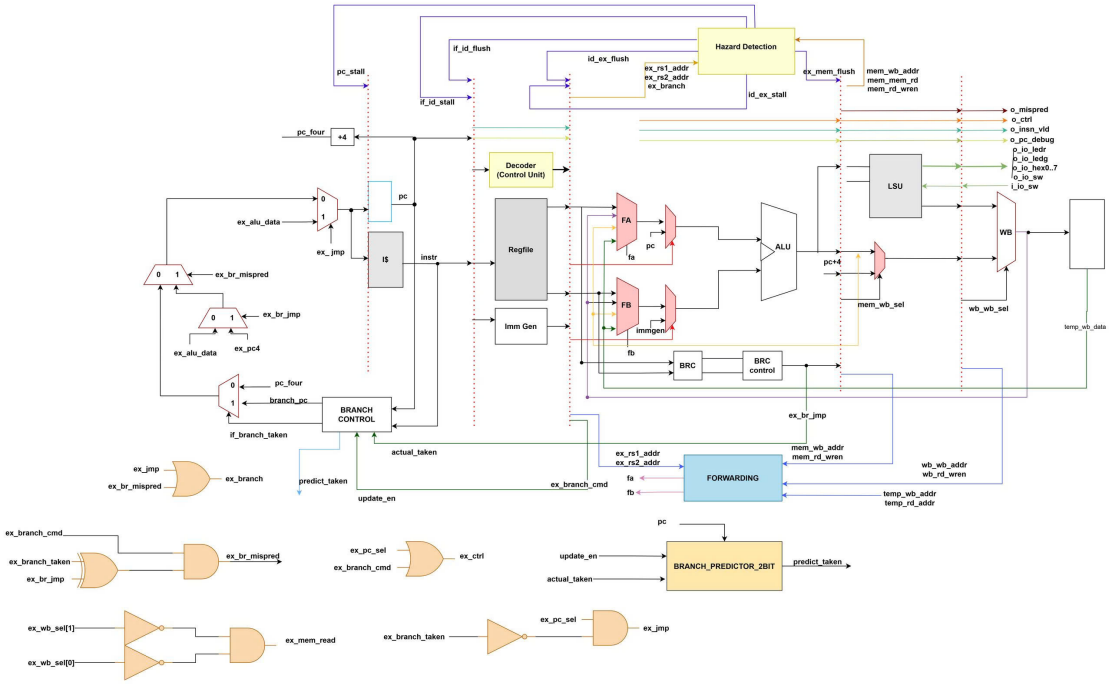
Cụ thể, ở stage IF, `branch_control` đọc trực tiếp `if_instr` và `if_pc`: nếu là JAL (`i_instr[6:2]`==11011) hoặc B-type branch (`i_instr[6:2]`==11000) thì xuất `branch_taken`=1 và tính trước `branch_pc` = pc + imm (J-type/B-type). Khi đó luồng PC được ưu tiên chọn theo dự đoán: nếu không có chuyển luồng từ EX và không có sửa sai, thì `pc_next` sẽ đi đến `if_branch_pc` khi `if_branch_taken`=1, ngược lại mặc định `pc_next` = pc + 4.

Song song, nhóm vẫn quyết định nhánh "thật"ở EX: khối `brc_control` dùng toán hạng đã forwarding (`fw_a`, `fw_b`) để tạo `ex_br_jmp` (taken thực tế). Trạng thái dự đoán được pipeline qua `id_branch_taken` → `ex_branch_taken`, và nhóm phát hiện sai dự đoán bằng công thức:

$$\texttt{ex_br_mispred} = \texttt{ex_branch_cmd} \wedge (\texttt{ex_branch_taken} \oplus \texttt{ex_br_jmp})$$

Với predictor always-taken cho branch, mispredict chủ yếu xảy ra khi dự đoán taken nhưng thực tế not-taken; lúc này `pc_next` được sửa về `ex_pc4` (PC+4 của lệnh branch ở EX). Ngoài ra, JALR không được dự đoán ở IF nên nhóm xử lý bằng `ex_jmp` = (~`ex_branch_taken` & `ex_pc_sel`) để redirect PC sang `ex_alu_data` tại EX.

2.5 Model 4: Forwarding with 2-bit Scheme Prediction



Hình 5: *Datapath của Model 4 - Pipeline với Forwarding và 2-bit Prediction*

Model 4 nâng cấp lên dự đoán động bằng bộ đếm 2-bit. Bộ đếm có 4 mức (00: chắc chắn không nhảy, 01: có thể không nhảy, 10: có thể nhảy, 11: chắc chắn nhảy), cần 2 lần sai liên tiếp mới đổi dự đoán, giúp chống nhiễu tốt.

Module `branch_predictor_2bit` được gắn vào `branch_control`, dùng chung 1 bộ đếm cho tất cả các lệnh rẽ nhánh. Dự đoán (`predict_taken`) được tính tổ hợp từ bit cao của counter, trong khi counter được cập nhật đồng bộ ở cạnh lên clock. Ví dụ: nếu counter = 01 (có thể không nhảy) mà thực tế là nhảy, thì counter sẽ tăng lên 10 (đổi sang dự đoán nhảy). Với lệnh JAL, vẫn luôn dự đoán nhảy (giống Model 3), còn với lệnh branch thông thường thì dùng kết quả từ bộ đếm.

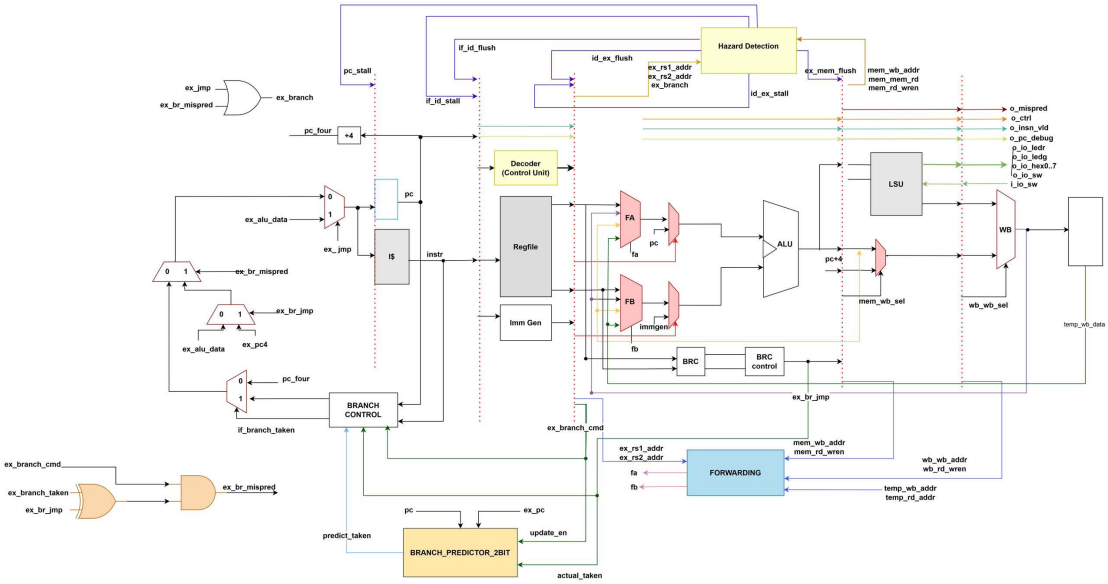
Cơ chế học (feedback loop): Khi lệnh branch đến stage EX, `brc_control` tính ra kết quả thực tế (`ex_br_jmp`) rồi gửi về `branch_predictor` để cập nhật bộ đếm. Quá trình này mất khoảng 2 cycle: branch ở IF tạo dự đoán, sau 2 cycle đến EX tính kết quả thực tế, rồi ở cycle tiếp theo bộ đếm được cập nhật. Nhờ vậy, bộ dự đoán tự động học từ hành vi thực tế của chương trình và điều chỉnh cho chính xác hơn.

Hiệu quả: Với vòng lặp chạy 100 lần (99 lần nhảy, 1 lần thoát), bộ đếm nhanh chóng ổn định ở mức 11 (chắc chắn nhảy) và chỉ đoán sai 1 lần cuối (độ chính xác 99%). Với pattern có nhiễu nhẹ (4 lần nhảy, 1 lần không, lặp lại), bộ đếm dao động giữa 10-11, độ chính xác khoảng 80%. Tuy nhiên, nếu pattern xen kẽ đều (nhảy-không-nhảy-không...), bộ đếm dao động liên tục và độ chính xác chỉ còn 12.5% (tệ hơn always-taken) vì chỉ có 1 bộ đếm dùng chung cho mọi branch (gọi là vấn đề aliasing).

Model này phù hợp với chương trình có 1 vòng lặp chính, pattern ổn định (nhảy >70% hoặc không nhảy >70%), nhưng kém hiệu quả với chương trình có nhiều branch khác nhau (bị aliasing)

hoặc pattern xen kẽ không đều.

2.6 Model 5: Forwarding with PC-indexed 2-bit Prediction



Hình 6: *Datapath của Model 5 - Forwarding with PC-indexed 2-bit Prediction*

Model 5 kế thừa toàn bộ cơ chế *data forwarding* và *2-bit branch prediction* của Model 4, nhưng được nâng cấp lên kiến trúc Harvard bằng cách tách riêng Instruction Memory (IMEM) ra khỏi Load-Store Unit (LSU), tạo thành đường lệnh và đường dữ liệu hoàn toàn độc lập. Đồng thời, branch predictor được mở rộng thành bảng dự đoán với 1024 entries được đánh chỉ mục trực tiếp theo địa chỉ PC.

IMEM được thiết kế như một module độc lập, chỉ đọc, với dung lượng 65536 bytes. Instruction được đọc theo cách tổ hợp theo biểu thức

$$o_instr = d_mem[i_pc[15:2]]$$

trong đó PC được word-aligned bằng cách bỏ hai bit thấp nhất. LSU được đơn giản hóa bằng cách loại bỏ hoàn toàn chức năng instruction fetch, chỉ tập trung xử lý data memory và I/O. Nhờ đó, stage IF có thể đọc instruction từ IMEM đồng thời với stage MEM truy cập data memory mà không xảy ra xung đột, giúp tăng băng thông và loại bỏ logic arbitration cho bộ nhớ dùng chung.

Branch predictor trong Model 5 được triển khai dưới dạng bảng dự đoán (*Pattern History Table*) với 1024 entries, mỗi entry chứa một bộ đếm bão hòa 2-bit độc lập. Chỉ số index được tính trực tiếp từ PC theo công thức $index = pc[11:2]$. Tại stage IF, dự đoán nhánh được thực hiện theo cách tổ hợp với tín hiệu

$$predict_taken = counter[index][1]$$

tương ứng với bit MSB của bộ đếm. Khi lệnh branch được thực thi tại stage EX, kết quả nhánh thực tế `ex_br_jmp` được gửi về để cập nhật bộ đếm tại entry tương ứng với `ex_pc[11:2]` theo cơ chế saturating counter.

Kiến trúc PC-indexed này giải quyết vấn đề aliasing tồn tại trong Model 4, trong đó nhiều branch khác nhau có thể chia sẻ cùng một entry dự đoán. Trong Model 5, mỗi địa chỉ branch có entry riêng và học hành vi một cách độc lập, tránh nhiễu lẫn giữa các nhánh. Về hiệu năng, Model 5 đạt IPC và tỉ lệ misprediction tốt hơn Model 4, đặc biệt đối với các chương trình có mật độ branch cao và nhiều địa chỉ nhánh khác nhau.

Đánh đổi của Model 5 là tiêu tốn thêm logic elements cho bảng dự đoán (1024 × 2 bits, tương đương 2048 bits lưu trữ) và không hỗ trợ *self-modifying code*, do IMEM được thiết kế là vùng chỉ đọc trong quá trình thực thi.

3 VERIFICATION STRATEGY

3.1 Kiểm tra hoạt động CPU với từng câu lệnh

Kiểm tra hoạt động của từng model với ISA test để xác nhận tất cả lệnh RV32I hoạt động chính xác, bao gồm lệnh số học, logic, shift, load/store, branch và jump.

Kết quả ISA Test cho 4 models:

Chỉ số	**Model 1 Non-fwd**	**Model 2 Forward**	**Model 3 Always-T**	**Model 4 2-bit**	**Model 5 Gshare**
Total Clock Cycles	12,282	7,830	6,564	6,312	6,350
Total Instructions	4,958	4,825	4,625	4,825	4,825
Total Branch Instr.	1,528	1,602	1,602	1,602	1,602
Total Mispredictions	1,380	1,449	816	690	709
IPC	**0.37**	**0.62**	**0.74**	**0.76**	**0.76**
Mispred. Rate	**90.31%**	**90.45%**	**50.94%**	**43.07%**	**44.26%**

Bảng 2: Kết quả ISA Test - Dữ liệu thực từ Simulation

Bảng trên tổng hợp kết quả thực nghiệm từ ISA test cho 4 models. Có thể thấy rõ sự cải thiện về IPC từ Model 1 (0.37) lên Model 4 (0.76), tương đương tăng 105%. Branch misprediction rate cũng giảm đáng kể từ 90.31% xuống 43.07%, chứng minh hiệu quả của forwarding và branch prediction. Model 4 hoàn thành test nhanh nhất với chỉ 6,312 cycles, giảm 48.6% so với Model 1. Model 5 tiếp tục cải thiện với IPC đạt 0.76 và branch misprediction rate giảm xuống 44.26%, hoàn thành test trong 6,350 cycles, cho thấy sự tối ưu hóa ổn định của kiến trúc pipeline.

3.2 Kiểm tra IPC và Misprediction với Benchmarks

Nhóm viết các chương trình benchmark để kiểm tra IPC và tỉ lệ misprediction trong các tình huống khác nhau.

Benchmark 1: 2 câu lệnh rẽ nhánh taken và 1 non-taken (sử dụng x5)

```
lui x5, 0x00002
addi x5, x5, 0
addi x9, x0, 1
test_loop:
    addi x5, x5, -1
    nop
    beq x5, x0, target
    bne x5, x0, target
target:
    bge x5, x0, test_loop
ebreak
```

Benchmark 2: 1 câu lệnh taken và 2 non-taken (sử dụng x6)

```
lui x6, 0x00002
addi x6, x6, 0
test_loop:
    addi x6, x6, -1
    beq x6, x0, target
    beq x6, x0, target
    nop
target:
    bge x6, x0, test_loop
ebreak
```

Benchmark 3: 1 non-taken và 2 JAL (sử dụng x7)

```
lui x7, 0x00002
addi x7, x7, 0
test_loop:
    addi x7, x7, -1
    beq x7, x0, exit_point
    jal x0, target
    nop
target:
    jal x0, test_loop
exit_point:
    ebreak
```

Benchmark 4: 1 non-taken và 2 JALR (sử dụng x8)

```
lui x8, 0x00002
addi x8, x8, 0
test_loop:
    addi x8, x8, -1
    beq x8, x0, exit_point
    jalr x0, x0, target
    nop
target:
    jalr x0, x0, test_loop
exit_point:
    ebreak
```

4 RESULT

4.1 Quartus Synthesis Results

Model	Logic Elements	Utilization	Fmax (MHz)
Non-forwarding	2,485	7%	60.5
Forwarding	2,943	9%	55.7
Always Taken	3,078	10%	52
2-bit Prediction	3,098	10%	51.5
PC-index	8,900	26%	51

Bảng 3: Tổng hợp kết quả Synthesis trên Quartus

Kết quả tổng hợp trên Quartus cho thấy sự đánh đổi giữa hiệu suất và diện tích phần cứng. Model 1 đơn giản nhất với 2,485 logic elements và Fmax cao nhất 60.5 MHz. Model 2 thêm

forwarding unit tăng lên 2,943 LEs, Fmax giảm xuống 55.7 MHz. Model 3 và 4 với branch prediction sử dụng xấp xỉ 3,078-3,098 LEs, Fmax khoảng 51-52 MHz. Tất cả models đều đủ ổn định để chạy trên FPGA ở 50 MHz.

4.2 Performance Analysis

4.2.1 Nhận xét về IPC (Instructions Per Cycle)

Model 1 (Non-forwarding): IPC = 0.37 - Thấp nhất do phải stall nhiều lần. Với 4,958 instructions thực thi cần 12,282 cycles, tương đương mỗi instruction mất 2.48 cycles.

Model 2 (Forwarding): IPC = 0.62 - Cải thiện 68% so với Model 1. Số cycles giảm từ 12,282 xuống 7,830 (giảm 36%). Forwarding giải quyết hiệu quả RAW data hazard.

Model 3 (Always Taken): IPC = 0.74 - Tăng 19% so với Model 2. Với 6,564 cycles cho 4,625 instructions, giảm đáng kể stall từ branch misprediction.

Model 4 (2-bit Scheme): IPC = 0.76 - Cao nhất, tăng 105% so với Model 1 và 23% so với Model 2. Chỉ cần 6,312 cycles, giảm 48.6% thời gian so với Model 1.

Model 5 (2-bit Prediction + Separate IMEM): IPC = 0.76 – Không đổi so với Model 4 do logic pipeline và branch prediction giữ nguyên. Với 6,350 cycles cho 4,825 instructions, hiệu năng thực thi tương đương Model 4.

4.2.2 Nhận xét về Branch Misprediction Rate

Model 1 & 2: ∼90% miss rate - Predict-not-taken không phù hợp với ISA test có nhiều branch taken. 1,380/1,528 và 1,449/1,602 mispredictions.

Model 3: 50.94% miss rate - Giảm gần một nửa với 816/1,602 mispredictions. Xác nhận khoảng 50% branches trong ISA test là taken.

Model 4: 43.07% miss rate - Thấp nhất với 690/1,602 mispredictions. Giảm 52% so với Model 1 và 15% so với Model 3. Adaptive learning đạt accuracy 56.93%.

Model 5: 44.26% miss rate – Gần tương đương Model 4 với 709/1,602 mispredictions. Do sử dụng cùng cơ chế 2-bit saturating counter và thuật toán cập nhật giống hệt, độ chính xác dự đoán không cải thiện.

4.2.3 Tổng hợp và Kết luận

Kết quả thực nghiệm cho thấy sự đánh đổi rõ ràng giữa complexity và performance. Model 4 đạt hiệu suất cao nhất nhưng chỉ tốn thêm 5 logic elements so với Model 3 (2,939 vs 2,934), trong khi giảm misprediction từ 50.94% xuống 43.07%.

Về execution time, Model 4 nhanh gấp 1.95 lần Model 1 (6,312 vs 12,282 cycles), chứng minh việc đầu tư vào forwarding và branch prediction mang lại lợi ích đáng kể.

Trade-off chính là giữa area (logic elements) và performance (IPC, misprediction rate). Model 4 cung cấp best balance với chỉ 9% utilization nhưng đạt IPC cao nhất và misprediction thấp nhất. Tất cả models đều có Fmax $>$ 50 MHz, đủ để chạy ổn định trên FPGA.

(a) **(b)**

Hình 7: *Kết quả pass testcode*

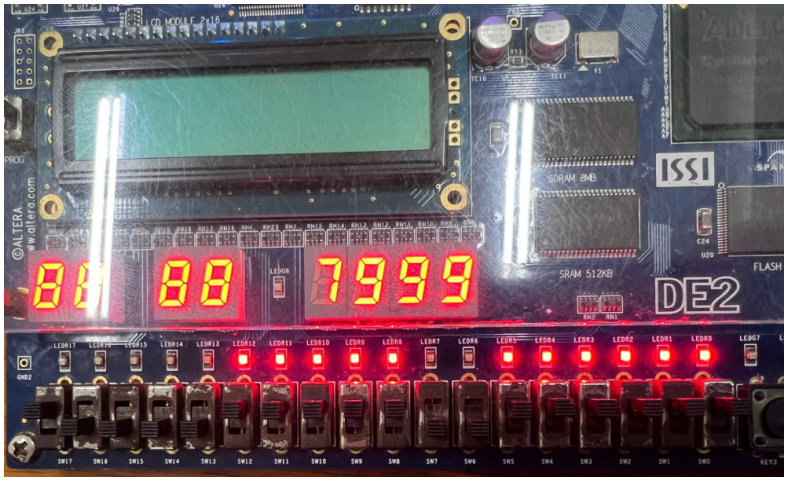
5 Applications

Để chứng minh khả năng hoạt động thực tế của bộ xử lý, nhóm đã phát triển các ứng dụng được viết bằng ngôn ngữ assembly RISC-V và kiểm thử thành công trên phần cứng FPGA kit DE2.

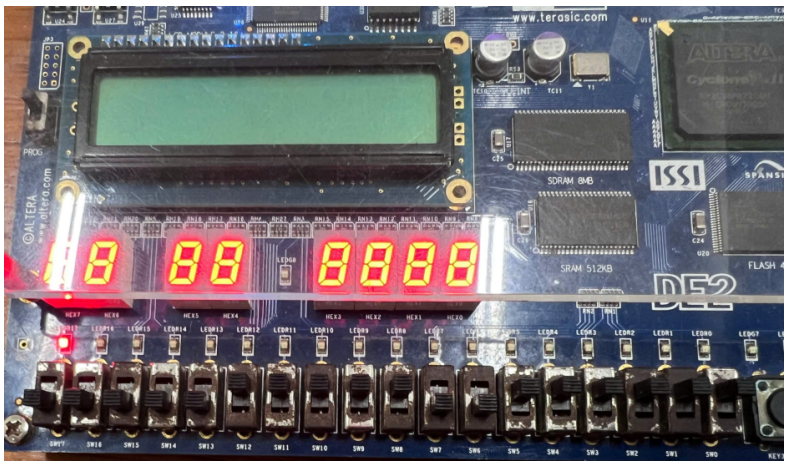
Một chương trình chuyển đổi số từ hệ Hexadecimal (13-bit) sang hệ Decimal được thiết kế để hiển thị kết quả lên 4 màn hình LED 7 đoạn. Ứng dụng có khả năng xử lý giá trị đầu vào từ 0 đến 8191 (0x0000 - 0x1FFF), tương ứng với không gian địa chỉ của 13 bit.

Ứng dụng được điều khiển thông qua các công tắc (switches) trên kit DE2, được ánh xạ tới địa chỉ I/O 0x10010000 của LSU:

1. SW[12:0] (13 switches): Nhập giá trị Hexadecimal cần chuyển đổi. Mỗi công tắc đại diện cho một bit trong giá trị 13-bit đầu vào.
   (a) Ví dụ: SW = 0b0_0000_0000_1010 (0x000A) sẽ hiển thị "0010"(decimal 10)
   (b) Ví dụ: SW = 0b1_1111_1111_1111 (0x1FFF) sẽ hiển thị "8191"(giá trị tối đa)
2. SW[17] (Reset switch): Hoạt động như tín hiệu reset không đồng bộ, xóa toàn bộ màn hình LED 7 đoạn về trạng thái "0000"khi được bật.



Hình 8: *Ứng dụng LED7D*



Hình 9: *SW 17 được bật lên reset LED*

Video demo kết quả chuyển đổi Hex sang Decimal: Xem tại đây

Tài liệu tham khảo

[1] Patterson, D. A., & Hennessy, J. L. (2017). Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition. Morgan Kaufmann.

[2] Waterman, A., & Asanović, K. (Eds.). (2019). The RISC-V Reader: An Open Architecture Atlas. Strawberry Canyon LLC.

[3] Harris, S. L., & Harris, D. M. (2021). Digital Design and Computer Architecture: RISC-V Edition. Morgan Kaufmann.

[4] RISC-V Foundation. (2023). RISC-V Specifications: Volume I – User-Level ISA. Retrieved from https://riscv.org/technical/specifications/